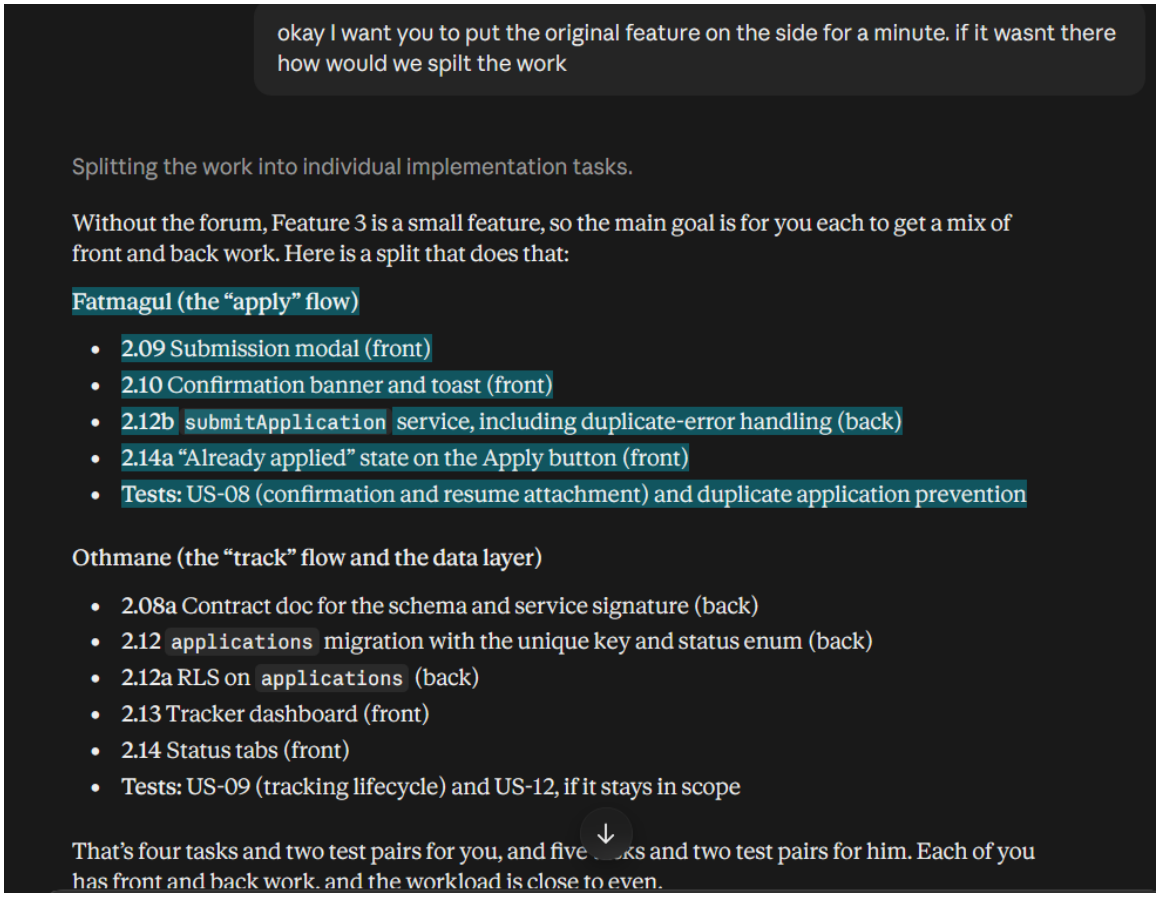
Task ID/Title: Feature 3 (not a specific task)

Purpose of AI Use: Splitting tasks fairly between teammates for Feature 3

Prompt/Response:



AI-Suggested Content: A mix of front-end and back-end features for each member explained in detail including tests for the feature

Feature 3 Task Split Fatmagul and Othmane.pdf

Validation: **manual review and peer review**, I verified the requirements of the feature and shared the list for my teammate to validate the output.

Decision: Accepted, but the contract was not written we just discussed in a meeting

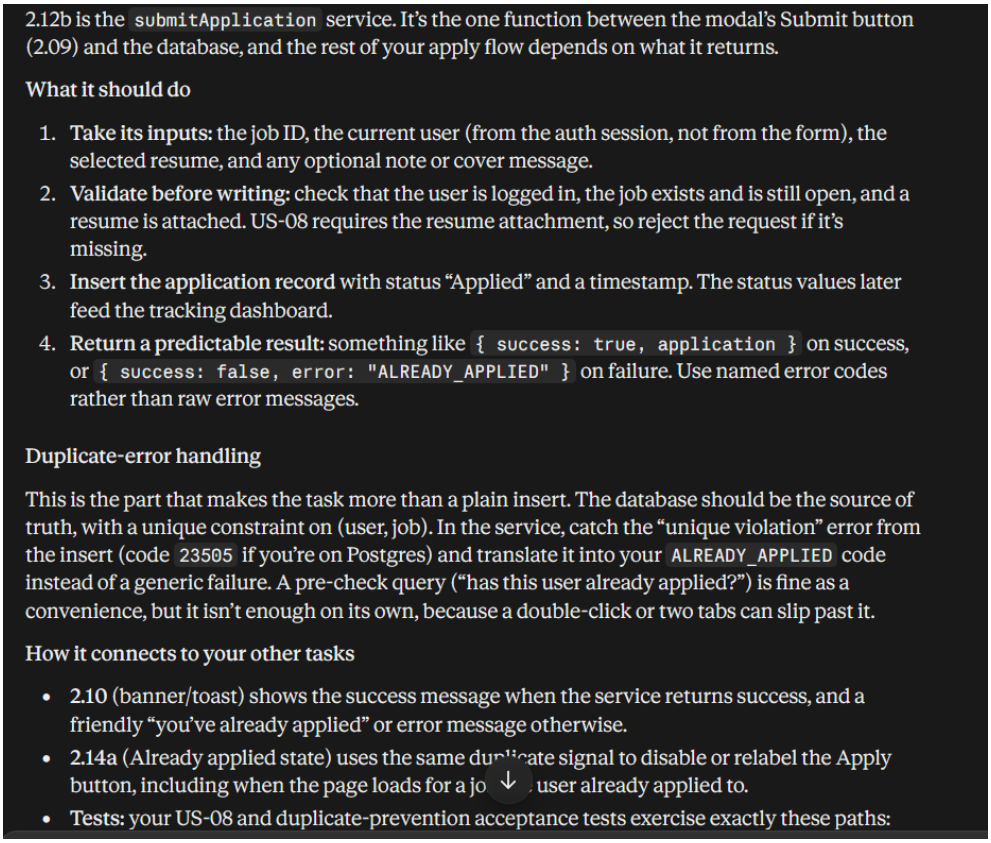
Reflection: Ai was useful to judge the weight of the tasks and splitting fairly

Responsible Person: Fatmagul Dedek

Task ID/Title: [2.12???] submitApplication service, including duplicate-error handling

Purpose of AI Use: Breakdown of task (pre-coding) and coding for SQL

Prompt/Response: Explain what I need to do to complete task 2.12b



2.12b is the `submitApplication` service. It's the one function between the modal's Submit button (2.09) and the database, and the rest of your apply flow depends on what it returns.

What it should do

1. **Take its inputs:** the job ID, the current user (from the auth session, not from the form), the selected resume, and any optional note or cover message.
2. **Validate before writing:** check that the user is logged in, the job exists and is still open, and a resume is attached. US-08 requires the resume attachment, so reject the request if it's missing.
3. **Insert the application record** with status "Applied" and a timestamp. The status values later feed the tracking dashboard.
4. **Return a predictable result:** something like `{ success: true, application }` on success, or `{ success: false, error: "ALREADY_APPLIED" }` on failure. Use named error codes rather than raw error messages.

Duplicate-error handling

This is the part that makes the task more than a plain insert. The database should be the source of truth, with a unique constraint on (user, job). In the service, catch the "unique violation" error from the insert (code `23505` if you're on Postgres) and translate it into your `ALREADY_APPLIED` code instead of a generic failure. A pre-check query ("has this user already applied?") is fine as a convenience, but it isn't enough on its own, because a double-click or two tabs can slip past it.

How it connects to your other tasks

- **2.10** (banner/toast) shows the success message when the service returns success, and a friendly "you've already applied" or error message otherwise.
- **2.14a** (Already applied state) uses the same duplicate signal to disable or relabel the Apply button, including when the page loads for a jo user already applied to.
- **Tests:** your US-08 and duplicate-prevention acceptance tests exercise exactly these paths:

What should be changed in the supabase db?

Applications applicant info · SQL Copy

```sql
-- Run once in the Supabase SQL editor (Othmane owns the applications table).
-- Adds the applicant details, skills and the cover letter PDF path to each application.
-- Nullable so existing rows stay valid. The existing RLS policies still apply:
-- seekers insert/read their own rows, recruiters read applications to their jobs.

ALTER TABLE public.applications
    ADD COLUMN IF NOT EXISTS first_name TEXT,
    ADD COLUMN IF NOT EXISTS last_name TEXT,
    ADD COLUMN IF NOT EXISTS email TEXT,
    ADD COLUMN IF NOT EXISTS phone TEXT,
    ADD COLUMN IF NOT EXISTS degree TEXT,
    ADD COLUMN IF NOT EXISTS country TEXT,
    ADD COLUMN IF NOT EXISTS authorized_to_work_ca BOOLEAN,
    ADD COLUMN IF NOT EXISTS requires_sponsorship BOOLEAN,
    ADD COLUMN IF NOT EXISTS skills TEXT[],
    ADD COLUMN IF NOT EXISTS cover_letter_path TEXT;

```

How do I change the content in supabase?

1. Go to **supabase.com/dashboard** and open your project (the one whose URL is in `frontend/.env.local`).
2. In the left sidebar, click **SQL Editor**.
3. Click **New query**.
4. Open `applications-applicant-info.sql`, copy everything, and paste it into the editor.
5. Click **Run** (or press `Ctrl+Enter`). You should see "Success. No rows returned."

To check it worked, open **Table Editor**, select the `applications` table, and confirm the new columns are there: `first_name`, `last_name`, `email`, `phone`, `degree`, `country`, `authorized_to_work_ca`, `requires_sponsorship`, `skills` and `cover_letter_path`.

The script is safe to run more than once, so you can re-run the whole file if you ran an earlier version. Existing rows are unaffected, since the new columns are empty for them.

If you can't see the project or the SQL Editor, you don't have a role on it. Ask whoever created the Supabase project to add you, or ask Othmane to run the file, since he owns that table.

If the app later says a column is "not found in the schema cache", run this in the SQL editor and try again:

```sql
NOTIFY pgrst, 'reload schema';
```

If the cover letter upload fails with `upload_failed` ↓ e you test, check **Storage → Policies** for the `resumes` bucket. It needs to let a logged-in user upload into their own folder.

AI-Suggested Content: Analysis of the requirements for the task at hand. Steps that should be accomplished in order for the information in the front end to be logged in the database. How to navigate supabase and write the SQL lines for the information on the application submission.

Validation: manual review

Decision: Accepted

Reflection: Useful to better understand the process between the frontend button click and the database submission. Helps navigate Supabase and write the SQL prompts. Also reminds me of updating the files that reference the database. The website was tested with these updates and everything worked.

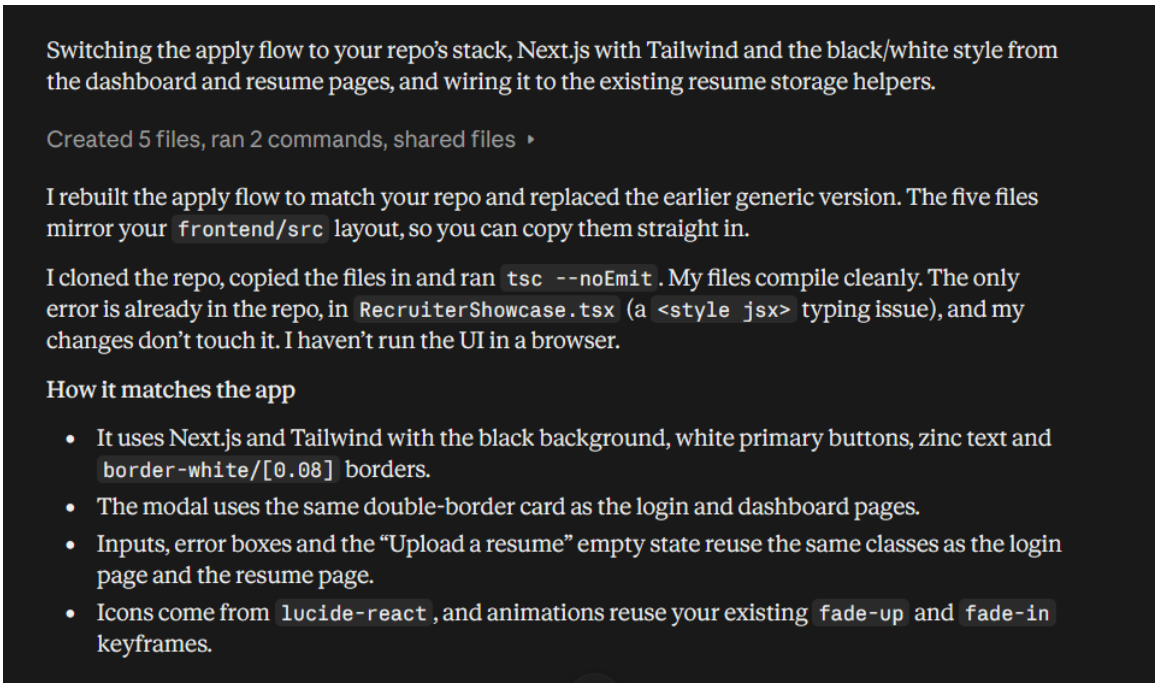
Responsible Person: Fatmagul Dedek

Task ID/Title: Submission modal and confirmation banner

Purpose of AI Use: Match the aesthetic of the code to what was already done in the previous sprint (coding)

Prompt/Response:

https://github.com/sudhansu-mohanty/SOEN341-launchpadai.git keep the same aesthetics



Switching the apply flow to your repo's stack, Next.js with Tailwind and the black/white style from the dashboard and resume pages, and wiring it to the existing resume storage helpers.

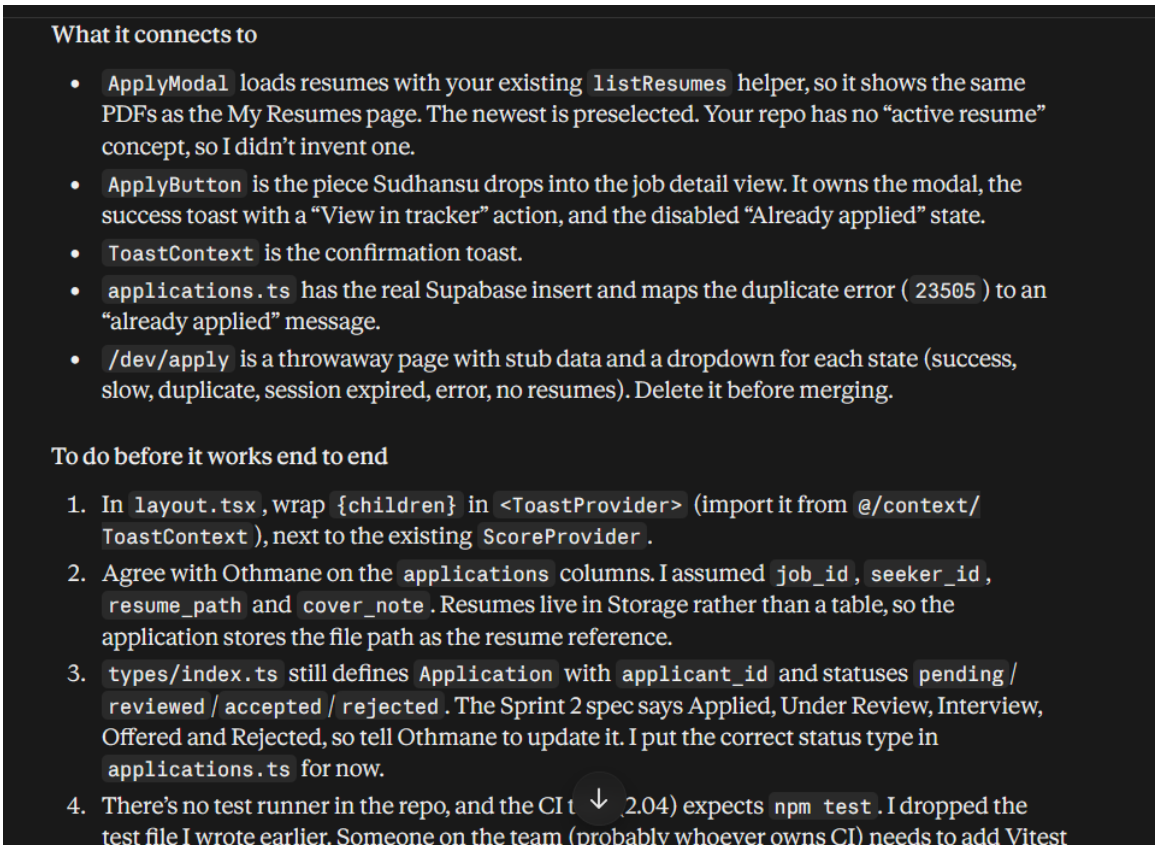
Created 5 files, ran 2 commands, shared files ▸

I rebuilt the apply flow to match your repo and replaced the earlier generic version. The five files mirror your `frontend/src` layout, so you can copy them straight in.

I cloned the repo, copied the files in and ran `tsc --noEmit`. My files compile cleanly. The only error is already in the repo, in `RecruiterShowcase.tsx` (a `<style jsx>` typing issue), and my changes don't touch it. I haven't run the UI in a browser.

How it matches the app

- It uses Next.js and Tailwind with the black background, white primary buttons, zinc text and `border-white/[0.08]` borders.
- The modal uses the same double-border card as the login and dashboard pages.
- Inputs, error boxes and the "Upload a resume" empty state reuse the same classes as the login page and the resume page.
- Icons come from `lucide-react`, and animations reuse your existing `fade-up` and `fade-in` keyframes.



What it connects to

- `ApplyModal` loads resumes with your existing `listResumes` helper, so it shows the same PDFs as the My Resumes page. The newest is preselected. Your repo has no "active resume" concept, so I didn't invent one.
- `ApplyButton` is the piece Sudhansu drops into the job detail view. It owns the modal, the success toast with a "View in tracker" action, and the disabled "Already applied" state.
- `ToastContext` is the confirmation toast.
- `applications.ts` has the real Supabase insert and maps the duplicate error (`23505`) to an "already applied" message.
- `/dev/apply` is a throwaway page with stub data and a dropdown for each state (success, slow, duplicate, session expired, error, no resumes). Delete it before merging.

To do before it works end to end

1. In `layout.tsx`, wrap `{children}` in `<ToastProvider>` (import it from `@/context/ToastContext`), next to the existing `ScoreProvider`.
2. Agree with Othmane on the `applications` columns. I assumed `job_id`, `seeker_id`, `resume_path` and `cover_note`. Resumes live in Storage rather than a table, so the application stores the file path as the resume reference.
3. `types/index.ts` still defines `Application` with `applicant_id` and statuses `pending` / `reviewed` / `accepted` / `rejected`. The Sprint 2 spec says Applied, Under Review, Interview, Offered and Rejected, so tell Othmane to update it. I put the correct status type in `applications.ts` for now.
4. There's no test runner in the repo, and the CI t ↓ 2.04) expects `npm test`. I dropped the test file I wrote earlier. Someone on the team (probably whoever owns CI) needs to add Vitest

AI-Suggested Content: Changed the files that deal with aspects of the submission modal so that it matches the colors and general aesthetics of the main page.

Validation: manual review

Decision: Accepted

Reflection: The AI successfully used the colors and visual elements of the main page. The modal blended seamlessly with the job searching page.

Responsible Person: Fatmagul Dedek

Task ID/Title: Acceptance tests writing

Purpose of AI Use: To get a general acceptance test structure

Prompt/Response:

How should the acceptance test be written for sprint 2

Per the Sprint 2 guidelines, you need **at least two acceptance tests for every user story from Sprint 1 and Sprint 2**. Each one must be a **GitHub issue, labeled, and linked to its user story and tasks.**

What makes a good acceptance test

An acceptance test checks that a user story works from the user's point of view. The standard format is Given/When/Then:

- **Given:** the starting state or preconditions
- **When:** the action the user takes
- **Then:** the observable, verifiable result

Aim for one happy path and one negative or edge case per story. That's an easy way to hit the two-test minimum with meaningful tests. Each test should be specific enough that a teammate or TA could run it and say pass or fail.

Suggested GitHub issue template

Title: `AT-12.1: Successful application submission shows confirmation`

Labels: `acceptance-test`, `sprint-2`, `feature-application` (plus a priority label if you use them)

Body:

- **Linked user story:** #12 (US-12: As a job seeker, I want to apply to a job so that I can be considered by the recruiter)
- **Related tasks:** #31, #32
- **Preconditions:** User is logged in and has an up ↓ ed resume. A job posting exists.
- **Steps:**

- Steps:
    1. Open the job detail page.
    2. Click “Apply”.
    3. Select a resume and submit.
- **Expected result:** A confirmation message appears, and the application shows in the dashboard with status “Applied”.
- **Status:** Not run / Pass / Fail

Example pairs for application tracking

US: Apply for a job

- AT-1 (happy path): Given a logged-in user with a resume, when they submit an application, then they see a confirmation and the status is “Applied”.
- AT-2 (negative): Given a user who already applied to a job, when they try to apply again, then the system blocks the duplicate and shows a message.

US: Track application status

- AT-1: Given a user with three applications, when they open the dashboard, then each application displays its current status (Applied, Under Review, Interview, Offered, or Rejected).
- AT-2: Given a recruiter updates a status to “Interview”, when the job seeker refreshes the dashboard, then the new status is shown.

AI-Suggested Content: Suggested structure with linked user story, related tasks, preconditions, steps, expected results and status.

Validation: manual review, comparing with the sprint 2 requirements

Decision: Accepted

Reflection: AI was useful to follow a proper template in order to write my acceptance tests as issues

Responsible Person: Fatmagul Dedek